Tutorial II.V - Dictionaries

Applied Optimization with Julia



Introduction

Imagine you have a school directory where each student's name is associated with their unique student ID. This is similar to how dictionaries work in programming - they allow you to store and retrieve information using key-value pairs.

Dictionaries are not just a generic programming topic: later in this course, they will store the data of our optimization models - for example, transport costs keyed by city, as in `costs = Dict("Hamburg" => 10.0, "Berlin" => 7.5)`. Time spent here pays off directly later.

Follow the structured instructions, implement your code in the designated blocks, and verify your implementations with `@assert` statements.

Section 1 - Creating and Accessing Dictionaries

Think of it this way:

- A dictionary is like a lookup table
- Each entry has a unique key (like a student's name)
- And an associated value (like their ID number)

Let's see some examples:

```
# Creating a dictionary
student_ids = Dict(
    "Elio" => 1001,
    "Mike" => 1002,
    "Yola" => 1003
)
```

```
Dict{String, Int64} with 3 entries:
  "Yola" => 1003
  "Elio" => 1001
  "Mike" => 1002
```

```
# Accessing values
println("Elio's ID: ", student_ids["Elio"])
```

```
Elio's ID: 1001
```

```
# Adding a new entry
student_ids["David"] = 1004
```

```
1004
```

```
# Checking if a key exists
if haskey(student_ids, "Eve")
    println("Eve's ID: ", student_ids["Eve"])
else
    println("Eve is not in the directory")
end
```

```
Eve is not in the directory
```

Instead of checking with `haskey` first, we can also use `get` to look up a key and provide a default value in case the key does not exist:

```
# Looking up a key with a default value
println("Eve's ID: ", get(student_ids, "Eve", "No ID found"))
```

```
Eve's ID: No ID found
```

```
# Removing an entry
delete!(student_ids, "David")
```

```
Dict{String, Int64} with 3 entries:
  "Yola" => 1003
  "Elio" => 1001
  "Mike" => 1002
```

Exercise 1.1 - Add to a Dictionary

Add a new book called “Harry Potter and the Philosopher’s Stone” with the author “J.K. Rowling” to the created dictionary.

```
# Creates a dictionary of books and authors
books = Dict(
    "1984" => "George Orwell",
    "Nexus" => "Yuval Noah Harari"
)
# YOUR CODE BELOW
```

```
# Test your answer
@assert haskey(books, "Harry Potter and the Philosopher's Stone")
```

```
@assert books["Harry Potter and the Philosopher's Stone"] == "J.K. Rowling"
println("Great! You've successfully added a new book to the books dictionary.")
```

Exercise 1.2 - Modify a Dictionary

Change the author of "1984" to "Eric Blair" (George Orwell's real name).

```
# YOUR CODE BELOW


# Test your answer
@assert books["1984"] == "Eric Blair"
println("Great! You've successfully modified the books dictionary.")
```

Section 2 - Advanced Dictionary Operations

Dictionaries can do more than just store simple information. Let's explore some features:

```
# A dictionary of student grades
grades = Dict(
    "Elio" => [85, 92, 78],
    "Mike" => [76, 88, 94],
    "Yola" => [90, 91, 89]
)
```

```
Dict{String, Vector{Int64}} with 3 entries:
  "Yola" => [90, 91, 89]
  "Elio" => [85, 92, 78]
  "Mike" => [76, 88, 94]
```

```
# Get all the keys (student names)
student_names = keys(grades)
println("Students: ", student_names)
```

```
Students: ["Yola", "Elio", "Mike"]
```

```
# Get all the values (grade lists)
all_grades = values(grades)
println("All grades: ", all_grades)
```

```
All grades: [[90, 91, 89], [85, 92, 78], [76, 88, 94]]
```

```
# Calculate average grade for each student
for (student, grade_list) in grades
    average = sum(grade_list) / length(grade_list)
    println("$student's average grade: $average")
end
```

```
Yola's average grade: 90.0
Elio's average grade: 85.0
Mike's average grade: 86.0
```

> **Note**
> The `(student, grade_list)` is a tuple that contains the key and value of each entry in the dictionary. We could also name the tuple as `(key, value)` or `(a, b)`.

Exercise 2.1 - Compute Averages into a New Dictionary

Compute the average grade for each student and store it in the dictionary `averages`. The keys should be the student names and the values their average grades. The next line initializes `averages` as an empty dictionary.

```
averages = Dict()
# YOUR CODE BELOW
```

```
# Test your answer
@assert averages["Elio"] == 85.0
@assert averages["Mike"] == 86.0
@assert averages["Yola"] == 90.0
println("Great! You've computed all averages: ", averages)
```

> **Tip**
> Loop over the key-value pairs of `grades` as shown in the example above. Inside the loop, you can add an entry to `averages` with `averages[student] = ...`.

Conclusion

Great! You've just navigated through the basics of dictionaries in Julia. Dictionaries are powerful data structures that allow for efficient data organization and retrieval. Continue to the next file to learn more advanced Julia concepts.

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography